udp Escuela de Informática y Telecomunicaciones
FACULTAD DE INGENIERÍA Y CIENCIAS

UNIVERSIDAD DIEGO PORTALES
ESCUELA DE INFORMÁTICA & TELECOMUNICACIONES

TALLER DE REDES Y SERVICIOS

Entrega Tarea 2: Capítulos I y II

Protocolo PostgreSQL (PSQL)

Autores:
Gustavo Fernandez
Alvaro Acevedo

Profesor:
Jonas Peñailillo

Fecha de entrega: 10 de 11 del 2026

Introducción

Los servicios de red se pueden estudiar observando el intercambio entre un cliente y un servidor. En este laboratorio se implementó una comunicación entre el servidor PostgreSQL y el cliente de consola psql, ejecutados en contenedores separados. Se generaron consultas SQL, respuestas con filas, cambios en datos y errores controlados, y se conservaron los paquetes para contrastar el comportamiento esperado con el observado.

El trabajo corresponde a la Tarea 2 de Taller de Redes y Servicios, semestre 2026-2, del grupo de Álvaro Acevedo y Gustavo Fernández. La selección del grupo aparece rotulada como “PSQL”; técnicamente, psql es el cliente y lo que se analiza en la red es el protocolo Frontend/Backend de PostgreSQL.

El objetivo es instalar y configurar ambos programas, comprobar su comunicación y explicar las fases y mensajes del tráfico generado. El desarrollo contiene el capítulo 1, Introducción al protocolo, y el capítulo 2, Análisis de tráfico. Las conclusiones, referencias y anexos complementan estos capítulos. El capítulo 3, Modificación de tráfico y análisis de métricas, queda reservado para la Tarea 3; aquí se formulan hipótesis, sin afirmar que se haya intervenido el tráfico.

Desarrollo

Capítulo 1. Introducción al protocolo

1.1. Objetivo y arquitectura

PostgreSQL es un sistema de gestión de bases de datos que presta servicios a aplicaciones cliente. psql permite iniciar una sesión, enviar sentencias SQL y visualizar sus resultados. El cliente no accede directamente a los archivos de datos: el proceso servidor recibe las solicitudes, ejecuta las operaciones y devuelve resultados o errores.

En el laboratorio se utilizó el protocolo PostgreSQL versión 3.0 sobre TCP. El servidor escucha en el puerto 5432; el cliente utiliza un puerto de origen temporal. La red bridge taller_redes comunica ambos contenedores y resuelve el nombre pg_server. Las direcciones observadas fueron 10.89.0.4 para el cliente y 10.89.0.2 para el servidor (figura A1). Las direcciones 192.168.1.10 y 192.168.1.20 de la tabla equipos son datos de ejemplo y no las IP de los contenedores.

1.2. Tráfico esperado

Se espera una apertura TCP con SYN, SYN/ACK y ACK. Sobre esa conexión, el cliente puede solicitar SSL, enviar un StartupMessage e intercambiar mensajes de autenticación. Al aceptar la sesión, el servidor informa parámetros y queda disponible para consultas [1].

Una consulta simple utiliza Query. Cuando devuelve filas, su respuesta incluye RowDescription, DataRow, CommandComplete y ReadyForQuery. La finalización voluntaria usa Terminate y después se cierra TCP [1, 2]. El número de mensajes PostgreSQL no tiene por qué coincidir con el número de paquetes: una respuesta TCP puede transportar varios mensajes de aplicación.

1.3. Autenticación y cifrado

SASL permite negociar un mecanismo de autenticación. En SCRAM-SHA-256, el cliente y el servidor intercambian desafíos y pruebas calculadas a partir de la contraseña; no se envía la contraseña literal como texto claro. El flujo exitoso incluye AuthenticationSASL, respuesta inicial, continuación, respuesta del cliente, finalización y AuthenticationOk [3].

Autenticarse con SCRAM no cifra las consultas ni sus resultados. El cifrado del canal es una función distinta. SSLRequest permite consultar si el servidor acepta iniciar TLS; la respuesta N indica que no lo hará. En este laboratorio SHOW ssl devolvió off y se observó N en la captura, de modo que las sentencias y los datos pudieron inspeccionarse sin descifrar TLS (figuras A3 a A5).

1.4. Errores y funcionamiento esperado

Un error SQL durante una consulta simple puede generar ErrorResponse y luego ReadyForQuery, permitiendo continuar la sesión. En cambio, un fallo de autenticación impide completar el inicio de sesión. Se eligieron dos pruebas para distinguir estas situaciones: división por cero, SQLSTATE 22012, y contraseña incorrecta, SQLSTATE 28P01 [2, 4].

La recuperación que se estudia corresponde a consultas independientes, con autocommit y fuera de una transacción explícita. No se generaliza a un bloque BEGIN que haya quedado en estado de transacción fallida.

Capítulo 2. Análisis de tráfico

2.1. Entorno e instalación

El laboratorio se ejecutó en Fedora con Podman 6.1.3, PostgreSQL y psql 16.15, tcpdump 4.99.6 y libpcap 1.10.7. Se empleó la imagen oficial postgres:16-alpine para los dos contenedores; cada uno ejecuta un programa distinto. pg_server ejecuta postgres; pg_client mantiene un proceso de espera y permite invocar psql mediante podman exec. La figura A1 documenta el entorno observado.

La red se creó con podman network create taller_redes. El servidor se creó con podman run -d, el nombre pg_server, la red compartida y las variables POSTGRES_USER=estudiante, POSTGRES_PASSWORD=clave_lab y POSTGRES_DB=taller. El volumen taller_datos se montó en /var/lib/postgresql/data. Estas variables inicializan un volumen nuevo; cambiarlas después no cambia automáticamente los usuarios ni las claves existentes [5].

El cliente se creó con el nombre pg_client, la misma red y --cap-add=NET_RAW. Se instaló tcpdump mediante apk add --no-cache tcpdump. El primer intento de captura falló por ausencia de NET_RAW incluso ejecutando como root; se recreó únicamente el cliente con ese permiso. El servidor y su volumen se mantuvieron. --cap-add permite agregar capacidades al contenedor [6].

Los registros de inicio muestran la preparación de la base, un servidor temporal y su posterior reinicio normal. La advertencia sobre locales no impidió que el servidor aceptara conexiones. La figura A2 muestra en Kitty los registros relevantes y una nueva inspección de la tabla. La captura de pantalla corresponde a esta comprobación posterior, no al momento original de creación.

Para reproducir el entorno, el repositorio incluye Containerfile.cliente y scripts/lab.py. El contenedor cliente reproducible incorpora tcpdump en su imagen. La imagen base se fija por digest al contenido observado; se registran las versiones, pero no se afirma que los repositorios de paquetes Alpine permanezcan inmutables.

2.2. Configuración y uso

La conexión se abrió con podman exec -it pg_client psql -h pg_server -U estudiante -d taller. La consulta de identificación confirmó la base taller, el usuario estudiante y el destino 10.89.0.2:5432. El usuario de laboratorio se crea con los privilegios iniciales de la imagen; no se configuró un despliegue de producción.

La tabla equipos contiene id entero autogenerado y clave primaria, nombre TEXT obligatorio y direccion_ip INET obligatoria. Se insertaron las filas servidor/192.168.1.10 y cliente/192.168.1.20 y se ejecutó SELECT * FROM equipos ORDER BY id. La figura A2 documenta la estructura y ambos registros. Los scripts SQL están en sql/ del repositorio. Las salidas completas se conservan como respaldo local.

2.3. Método de captura y trazabilidad

Se capturó en eth0 dentro de pg_client con tcpdump, sin modo promiscuo, con longitud completa de captura y filtro tcp port 5432. Para cada prueba automatizada se inició la captura antes de abrir psql y se detuvo con SIGINT después del cierre. La opción --immediate-mode evita que una sesión muy corta termine antes de que la captura entregue sus paquetes; -U vacía la salida por paquete. Los archivos se copiaron al equipo anfitrión y se analizaron con TShark, el analizador de consola de Wireshark.

La captura inicial manual, 00_consulta_inicial.pcap, se preservó sin cambios y tiene 33 paquetes y dos flujos TCP. El primer flujo termina después de que el servidor solicita autenticación; el segundo completa la sesión y contiene la consulta. No hay un ErrorResponse en el primer flujo, por lo que no se lo clasifica como contraseña incorrecta. La consulta está en el paquete 27 y su respuesta en el 28.

Las pruebas adicionales se conservaron en cuatro archivos separados. Sus cantidades, duración observada y tamaño de tramas se presentan en la tabla A1. Los registros de tcpdump reportaron cero paquetes descartados por el kernel en esas cuatro capturas; esto describe la captura local, no una prueba de ausencia de pérdidas en toda la red. Los hashes SHA-256 y los campos exportados permiten comprobar la identidad y el contenido de los archivos.

2.4. Sesión correcta: inicio y autenticación

En 01_sesion_completa.pcap, los paquetes 1 a 3 establecen TCP. El paquete 4 contiene SSLRequest, el 6 contiene la respuesta N y el 8 contiene StartupMessage, con usuario estudiante, base taller y aplicación psql (figura A3). Las pruebas automatizadas usan psql sin terminal interactiva. Sus parámetros de inicio pueden diferir de los de la captura manual. La tabla A2 ubica cada fase de la sesión completa.

Los paquetes 9 a 13 muestran la autenticación SCRAM-SHA-256. Se observa la selección SASL, el desafío y la respuesta, y luego la finalización y aceptación. El paquete 13 agrupa varios mensajes, incluidos ParameterStatus, BackendKeyData y ReadyForQuery (figura A4). No debe interpretarse que cada mensaje corresponde a una conexión distinta.

2.5. Consulta, resultados y cierre

El paquete 14 solicita identificar la conexión y el 15 responde. El paquete 16 contiene el SELECT sobre equipos. El paquete 17 agrupa RowDescription con tres columnas, dos DataRow, CommandComplete con SELECT 2 y ReadyForQuery. La figura A5 muestra en Kitty los tipos de mensajes extraídos del PCAP y una nueva ejecución del SELECT en psql, con ambas filas en texto legible.

El paquete 18 contiene Terminate y los paquetes 19 a 21 completan el cierre TCP con FIN/ACK, FIN/ACK y ACK. La sesión estudiada incluye apertura, autenticación, uso y cierre, y no solamente el contenido de una consulta aislada.

2.6. Operaciones de escritura y errores controlados

En 02_crud.pcap se creó o verificó una tabla auxiliar llamada pruebas_redes y se reinició solo esa tabla. Se insertaron dos filas, se consultaron, se actualizó el nombre del primer registro, se verificó el cambio, se eliminó el segundo registro y se consultó el resultado final. psql recibió INSERT 0 2, UPDATE 1 y DELETE 1. Quedó una fila con nombre router_actualizado (figura A6). La tabla equipos permaneció intacta.

Como la tabla auxiliar ya existía al repetir la prueba definitiva, CREATE TABLE IF NOT EXISTS produjo un NoticeResponse con SQLSTATE 42P07. Es un aviso de omisión de creación y no un error de la sesión. El respaldo local lo registra en evidencias/resumen.json bajo avisos.

En 03_error_sql.pcap, el paquete 14 envía SELECT 1 / 0. El 15 contiene ErrorResponse con SQLSTATE 22012 y mensaje division by zero. El 16 contiene ReadyForQuery. El cliente envía después otra consulta en el paquete 18 y el servidor devuelve conexion_sigue_activa en el 19. La sesión se conserva y termina voluntariamente (figura A7).

En 04_auth_fallida.pcap se utilizó una contraseña incorrecta, sin cambiar la del servidor. El paquete 13 devuelve FATAL con SQLSTATE 28P01. psql termina con código 2 y no aparecen mensajes Query en esa captura: no se permitió ejecutar SQL (figura A8). El cierre de esta conexión se diferencia de un error SQL dentro de una sesión ya autenticada.

2.7. Hipótesis sobre modificación del tráfico

Las siguientes son predicciones razonadas y no resultados experimentales. La Tarea 2 no incluyó edición de paquetes ni mediciones bajo tráfico modificado.

Cambiar la respuesta SSL de N a S sin habilitar TLS en el servidor haría que el cliente intentara negociar TLS con un extremo que espera otro formato. Se espera que la conexión falle antes de ejecutar consultas. Alterar StartupMessage, cambiando el usuario o la base por valores inexistentes y manteniendo válido el formato, debería provocar rechazo del inicio de sesión.

Modificar una prueba SCRAM sin conocer el secreto debería impedir su validación. Cambiar bytes de una consulta podría causar un error de sintaxis o una operación diferente si el resultado sigue siendo SQL válido. Alterar una longitud o el identificador de tipo podría desalinear el análisis de mensajes, producir un error de protocolo o cerrar la conexión. Cambiar datos en una respuesta válida podría modificar lo que visualiza el cliente sin modificar necesariamente lo almacenado en el servidor.

Estas hipótesis suponen que la intervención mantiene coherentes las longitudes, las secuencias TCP y las sumas de comprobación cuando corresponda. Una alteración arbitraria puede descartarse antes de que PostgreSQL la procese. Para la Tarea 3 se propone comparar sesiones normales e intervenidas mediante proporción de éxito, tiempo de respuesta, errores y retransmisiones, sin atribuir todavía efectos medidos.

Conclusiones

Se implementó una comunicación funcional entre psql y PostgreSQL en contenedores separados. La identificación de sesión y la consulta de equipos confirmaron el destino y el acceso a los datos. Las capturas muestran las etapas previstas: establecimiento TCP, negociación SSL rechazada, inicio de sesión, autenticación SCRAM, consultas, resultados y finalización.

Las pruebas controladas diferenciaron tres clases de respuesta: resultados con filas y confirmaciones de escritura, errores SQL recuperables dentro de una sesión y rechazo de autenticación antes de ejecutar consultas. La evidencia también permitió distinguir un aviso NoticeResponse de un ErrorResponse y observar que varios mensajes PostgreSQL pueden viajar dentro de un solo paquete TCP.

Las dificultades principales fueron el permiso NET_RAW para capturar y el vaciado de buffers en sesiones automatizadas muy breves. Se resolvieron agregando la capacidad al cliente y utilizando entrega inmediata de paquetes. La reproducción en un entorno separado creó los contenedores desde cero y ejecutó las mismas cuatro pruebas sin reutilizar la base del laboratorio principal.

El alcance se limita a un entorno local, con datos pequeños y sin TLS. Las duraciones y cantidades de la tabla A1 describen ejecuciones puntuales, no un estudio de rendimiento ni mediciones de latencia de PostgreSQL. No se ejecutaron modificaciones del tráfico. El video muestra la demostración del montaje con Podman en la red taller_redes. Los números de paquete del análisis corresponden a los PCAP indicados en cada sección.

Enlaces de entrega. Video de demostración en YouTube · Repositorio PSQL-TallerRedes en GitHub

Referencias

[1] PostgreSQL Global Development Group. PostgreSQL 16: Message Flow, sección 55.2. https://www.postgresql.org/docs/16/protocol-flow.html

[2] PostgreSQL Global Development Group. PostgreSQL 16: Message Formats, sección 55.7. https://www.postgresql.org/docs/16/protocol-message-formats.html

[3] PostgreSQL Global Development Group. PostgreSQL 16: SASL Authentication, sección 55.3. https://www.postgresql.org/docs/16/sasl-authentication.html

[4] PostgreSQL Global Development Group. PostgreSQL 16: Error Codes, apéndice A. https://www.postgresql.org/docs/16/errcodes-appendix.html

[5] Docker Official Images. postgres: documentación de la imagen oficial. https://hub.docker.com/_/postgres

[6] Proyecto Podman. podman-run: opciones de red, volúmenes y capacidades. https://docs.podman.io/en/v6.1.3/markdown/podman-run.1.html

Documentos del curso: Enunciado de Tarea 2, selección de protocolo del grupo y Template Informe Tareas Taller de Redes, suministrados en la carpeta de trabajo. Consulta de documentación técnica: 6 de noviembre de 2026, hora de Chile.

Anexos

A. Inventario de capturas y verificación

Tabla A1. Capturas reales. Bytes corresponde a la suma de frame.len, incluidas cabeceras; no es el tamaño del archivo PCAP. Duración corresponde al intervalo entre la primera y la última trama de cada archivo, no al tiempo de una consulta. Los recuentos pueden variar al reproducir.

Archivo PCAP	Paquetes	Flujos	Bytes	Intervalo (s)
00_consulta_inicial	33	2	3398	11.673280
01_sesion_completa	21	1	2753	0.013155
02_crud	35	1	4492	0.027189
03_error_sql	23	1	2589	0.010968
04_auth_fallida	16	1	1532	0.009295

Los cinco PCAP están en capturas/ del repositorio. Los hashes, las consultas, los avisos y las exportaciones de TShark se conservan como respaldo local en evidencias/. El PCAP de cada prueba permite revisar los paquetes originales en Wireshark.

Se ejecutaron 16 comprobaciones de integridad y comportamiento: capturas no vacías, cero descartes locales reportados, consulta visible, aceptación y cierre de sesión, SQLSTATE esperados, recuperación de consulta, ausencia de SQL en la conexión rechazada y confirmaciones de escritura. El resultado completo se conserva en el respaldo local evidencias/verificacion.json. La creación desde cero y las cuatro pruebas también se verificaron en un entorno separado. El registro se guarda en evidencias/reproduccion_desde_cero.txt del respaldo local.

Tabla A2. Fases y paquetes de la sesión completa

La numeración corresponde a 01_sesion_completa.pcap, que contiene 21 paquetes. En Wireshark, el filtro tcp.port == 5432 permite ver tanto los segmentos TCP de control como los mensajes PostgreSQL.

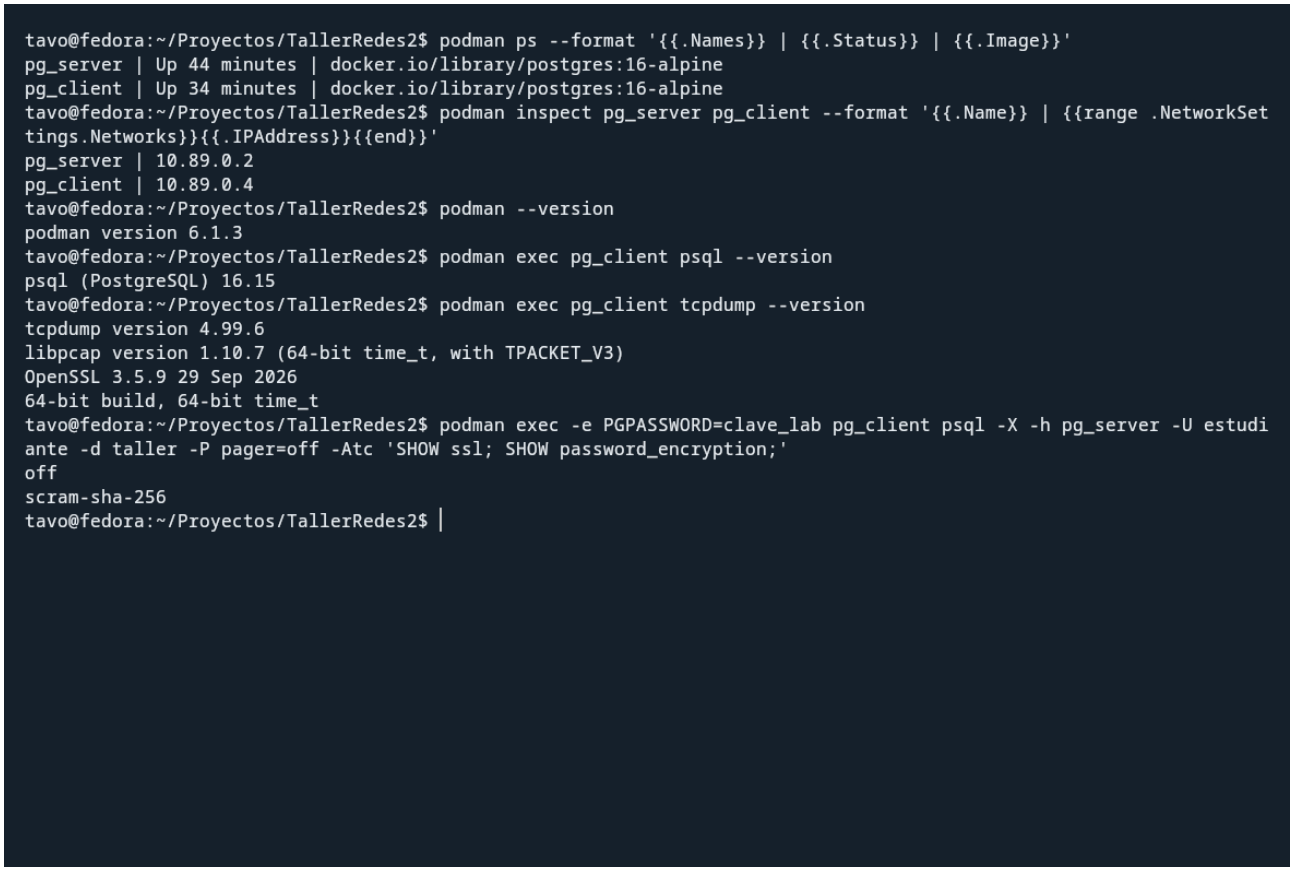
Fase	Paquetes	Evidencia observada
Establecimiento TCP	1-3	SYN, SYN/ACK y ACK abren la conexión.
Negociación SSL	4-7	SSLRequest en el 4 y respuesta N en el 6. Los paquetes 5 y 7 son ACK.
Inicio de sesión	8	StartupMessage informa el usuario estudiante, la base taller y la aplicación psql.
Intercambio SCRAM	9-12	Solicitud SASL, respuesta inicial, desafío y prueba del cliente.
Sesión lista	13	SASL final, AuthenticationOk, ParameterStatus, BackendKeyData y ReadyForQuery.
Identificación de la conexión	14-15	Consulta de base, usuario, IP y puerto, seguida de la respuesta.
Consulta de equipos	16-17	SELECT, descripción de columnas, dos DataRow, CommandComplete y ReadyForQuery.
Terminación	18-21	Terminate en el 18 y cierre TCP en los paquetes 19 a 21.

Las pruebas de división por cero y contraseña incorrecta se analizan por separado en 03_error_sql.pcap y 04_auth_fallida.pcap.

B. Evidencias visuales

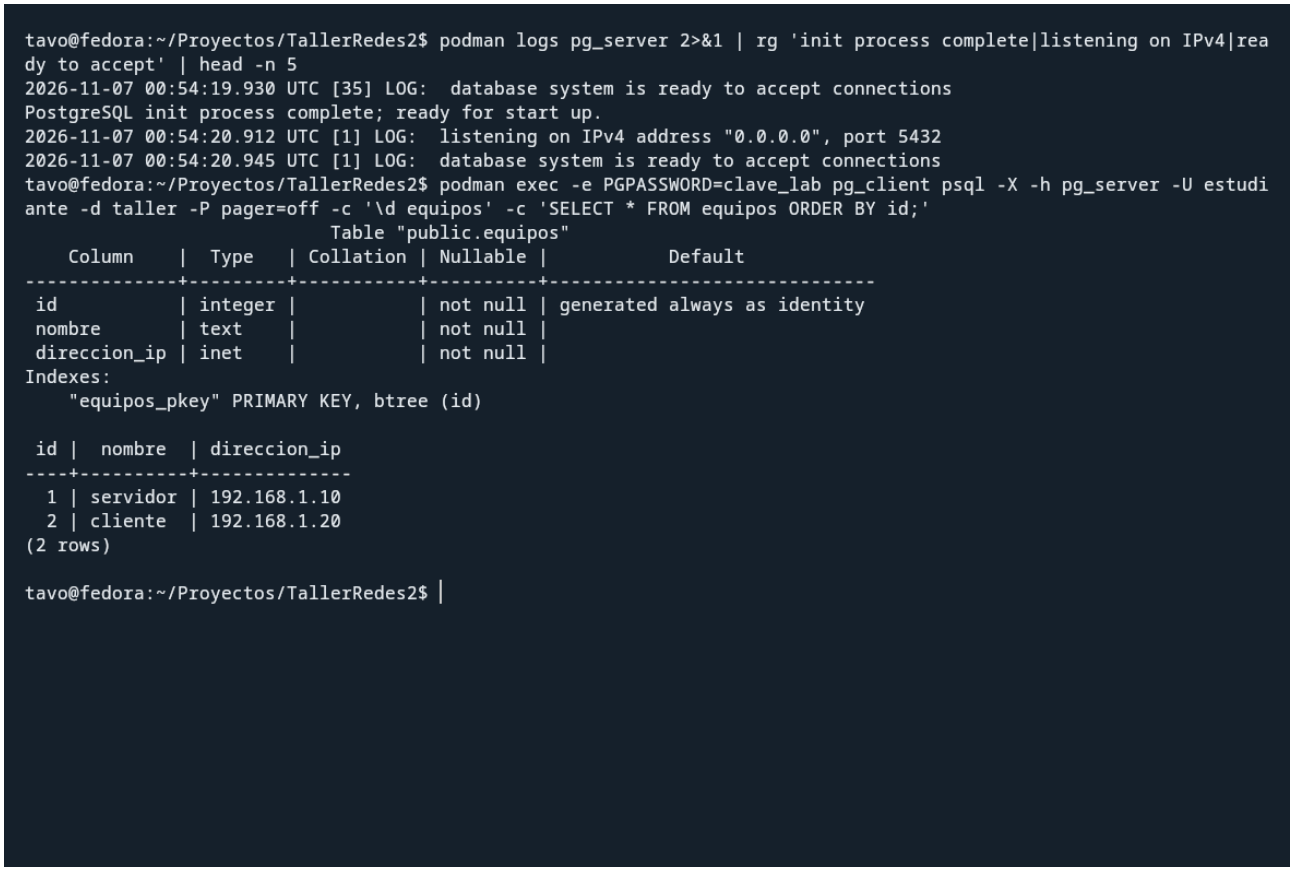
Las figuras siguientes son capturas de pantalla reales de una ventana de Kitty. Se ejecutaron nuevamente los comandos de comprobación y de uso de psql, y se consultaron los PCAP guardados mediante TShark. Las imágenes se tomaron durante una nueva comprobación del laboratorio. Los PCAP del análisis no se modificaron. Los textos de la consola y la procedencia de las figuras se conservan como respaldo local en evidencias/kitty/.

Figura A1. Configuración del laboratorio



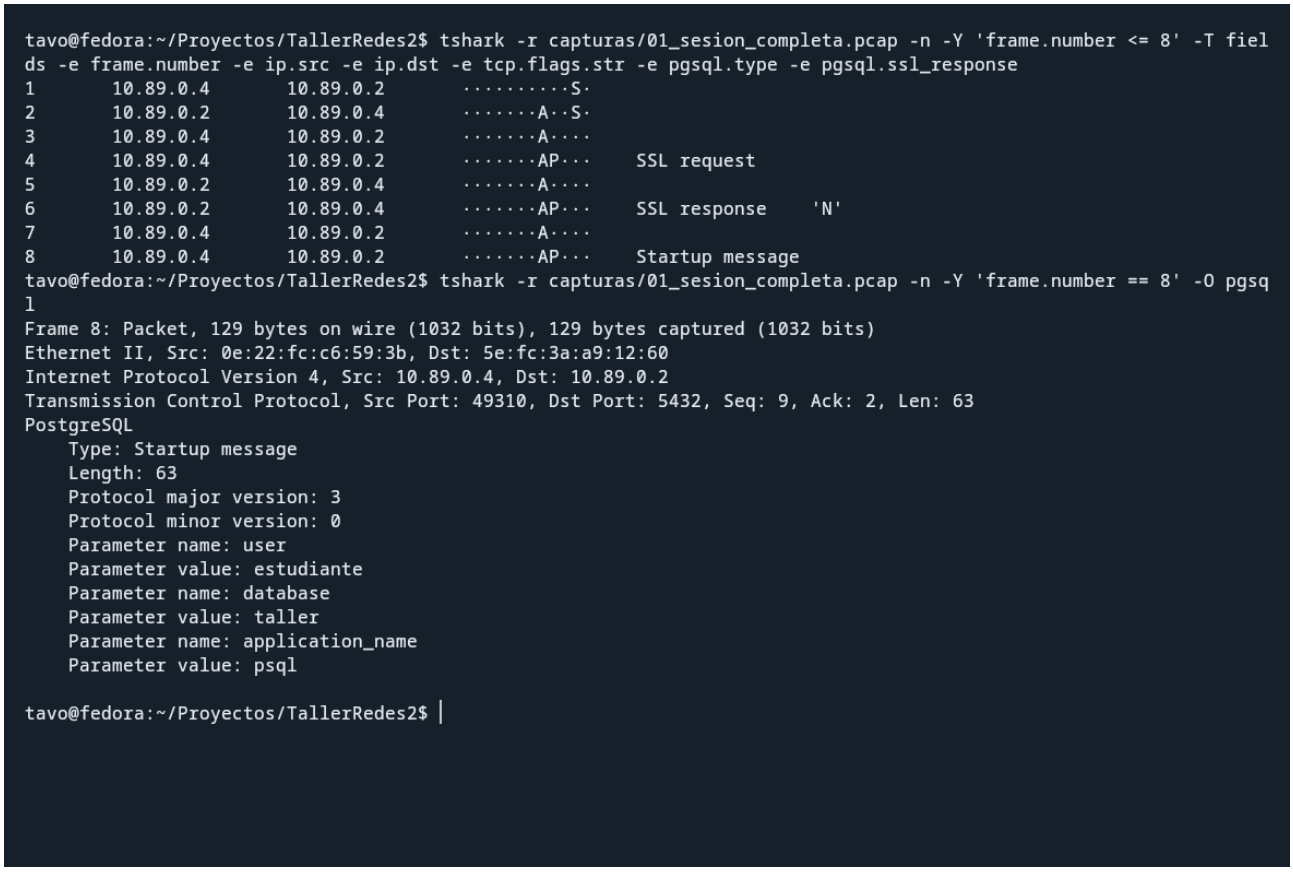
El cliente y el servidor comparten taller_redes. Solo el cliente requiere NET_RAW para capturar. El puerto 5432 se utiliza dentro de la red de contenedores, sin publicación al anfitrión.

Figura A2. Evidencia de inicio y datos iniciales



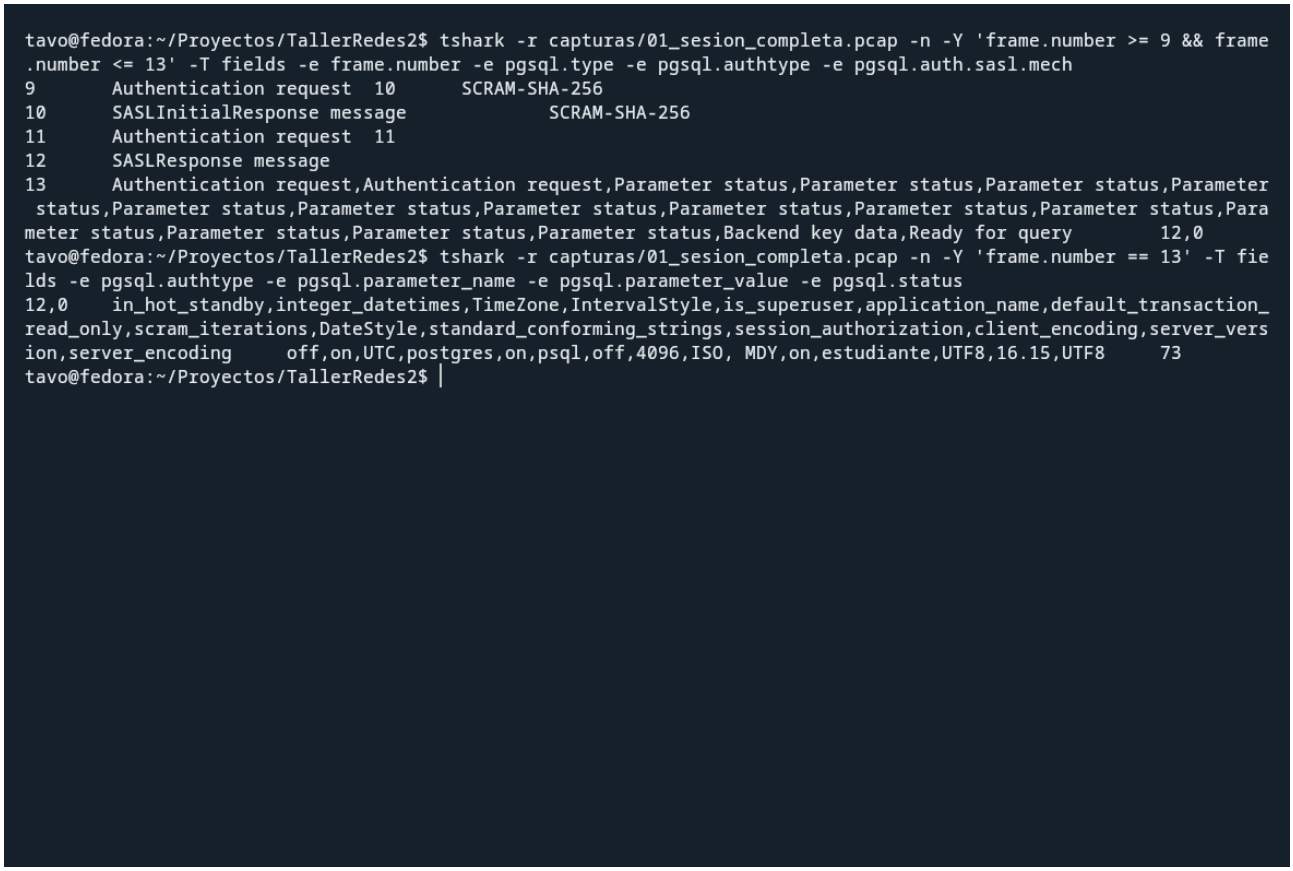
Captura real de Kitty: consulta de los registros de inicio y comprobación posterior de la estructura y los datos. La tabla conserva los dos registros ingresados durante la sesión guiada.

Figura A3. Establecimiento y mensajes de inicio



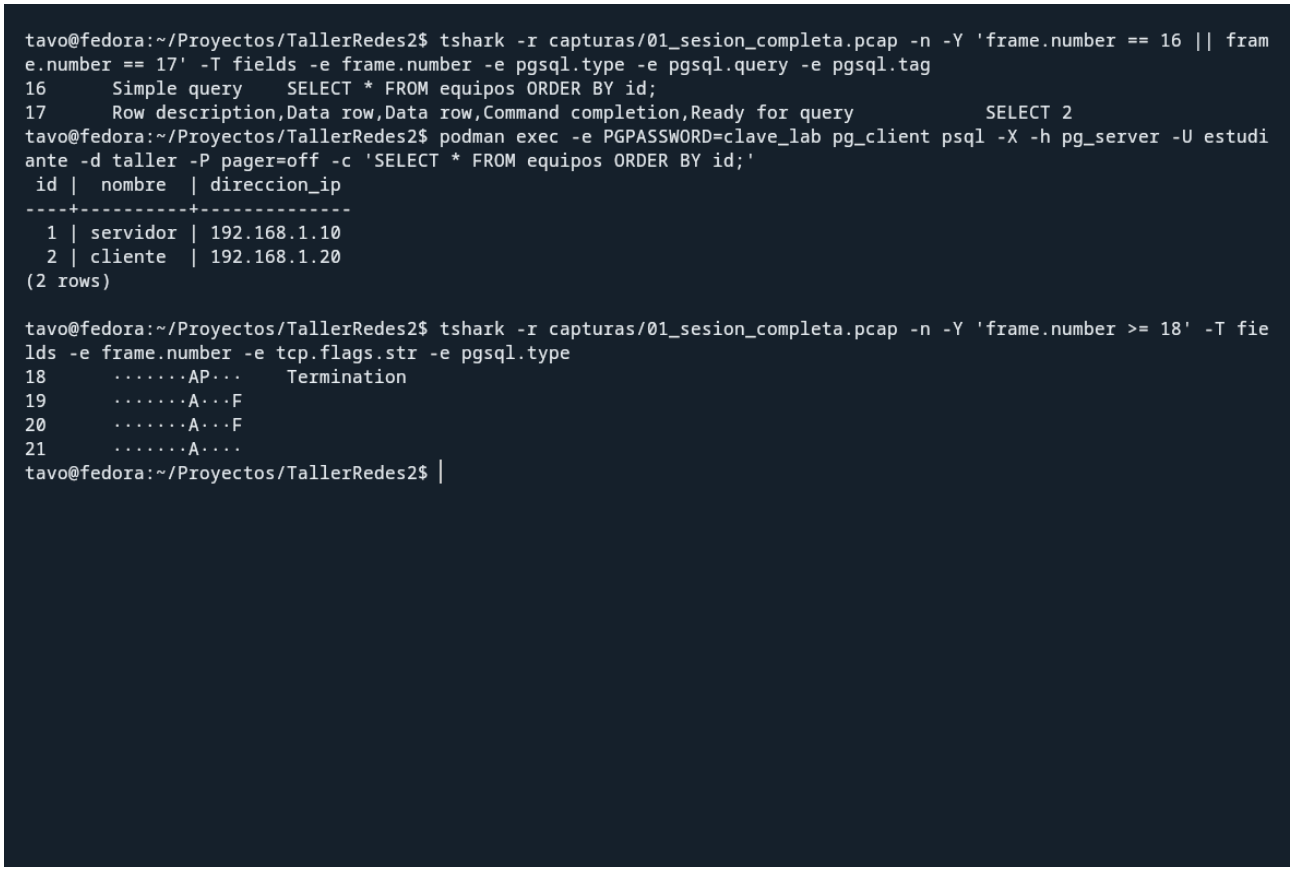
Se identifican la apertura TCP, SSLRequest, la respuesta N y los parámetros de StartupMessage. Los números remiten a 01_sesion_completa.pcap.

Figura A4. Autenticación correcta



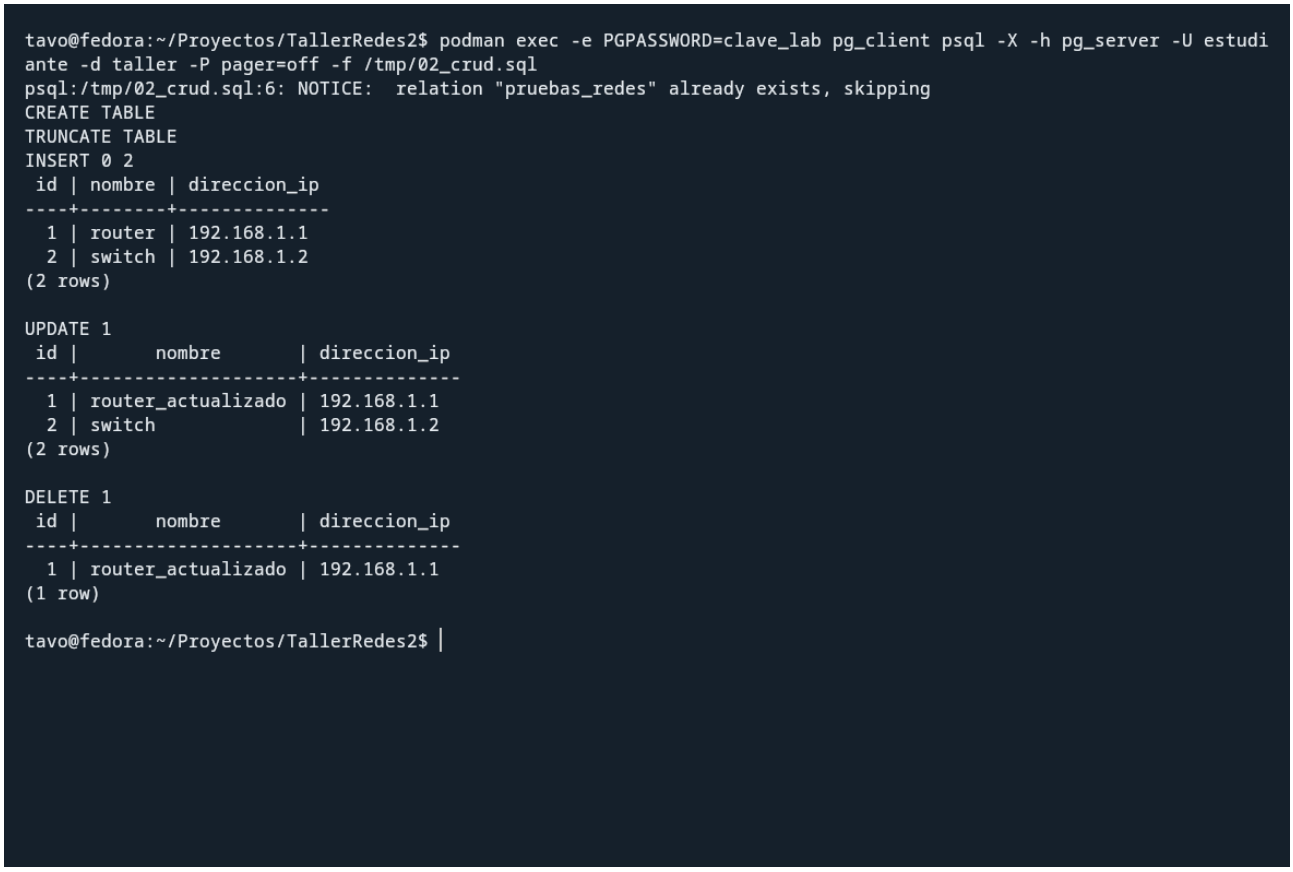
Captura real de Kitty con campos del PCAP: AuthenticationSASL (10), SASLContinue (11), SASLFinal (12) y AuthenticationOk (0). El paquete 13 también incluye parámetros y ReadyForQuery; el estado 73 corresponde a Idle. El detalle completo está en 01_sesion_completa_pgsql.txt.

Figura A5. Consulta, resultados y finalización



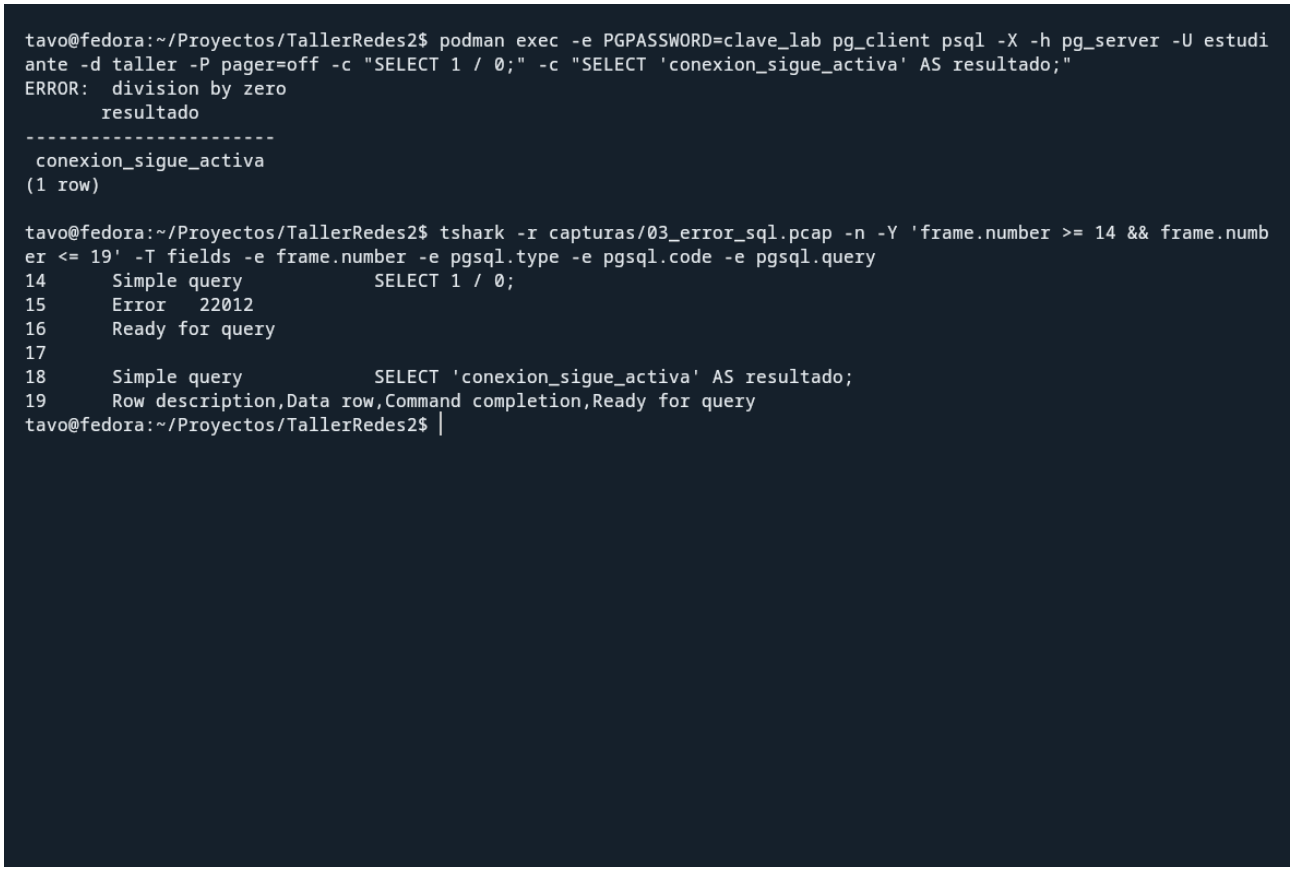
Captura real de Kitty con lectura de los paquetes 16 y 17, repetición del SELECT en psql y lectura del cierre. Las filas son visibles sin descifrar TLS. La consulta repetida es una comprobación posterior y no añade paquetes al PCAP guardado.

Figura A6. Inserción, actualización y eliminación



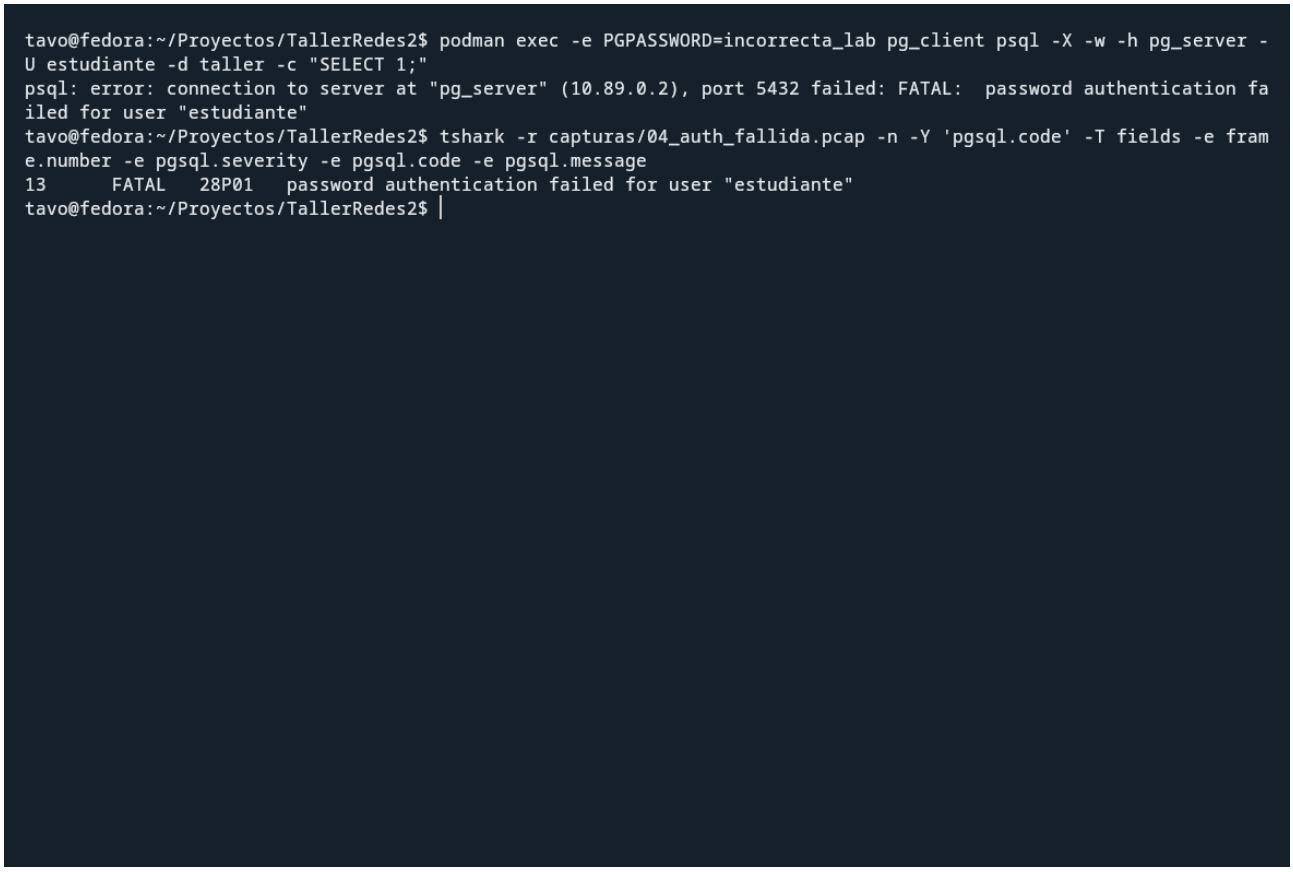
Captura real de una nueva ejecución de sql/02_crud.sql en Kitty, sobre pruebas_redes. Se observan las confirmaciones y una fila final. El aviso de tabla existente se debe a que la prueba es repetible; no impide continuar. No se modifica equipos.

Figura A7. Error SQL con recuperación



Captura real de Kitty: se repite el error y la consulta posterior en una misma sesión psql; luego se leen los campos del PCAP original. En esa captura, 22012 aparece en el paquete 15 y la respuesta a la segunda consulta en el 19.

Figura A8. Autenticación fallida



Captura real de Kitty: se repite el intento con contraseña incorrecta y se lee el rechazo del PCAP guardado. El servidor devuelve SQLSTATE 28P01 y severidad FATAL; no se llega a ejecutar la consulta SQL.